



Introducción a los Sistemas Distribuidos (75.43)

Trabajo Práctico N°1

File Transfer

1°C 2024

Integrantes del Grupo		
Nombre y apellido	Padrón	Correo
Rafael Berenguel	108225	rberenguel@fi.uba.ar
Juan Ignacio Biancuzzo	106005	jbiancuzzo@fi.uba.ar
Agustina Fraccaro	103199	afraccaro@fi.uba.ar
Sofia Javes	107677	sjaves@fi.uba.ar
Agustina Schmidt	103409	aschmidt@fi.uba.ar

Índice

1. Introducción	3
2. Hipótesis y suposiciones realizadas	3
3. Implementación	3
3.1. Protocolo RDT	3
3.1.1. Segmentos	4
3.1.2. Establecimiento de la conexión (Handshake)	5
3.1.3. Pérdida de paquetes	6
3.1.4. Stop and Wait	7
3.1.5. Selective Repeat	7
3.2. Protocolo de Transferencia de Datos	7
4. Pruebas	8
5. Preguntas a responder	9
6. Dificultades encontradas	10
7. Conclusión	10

1. Introducción

En el presente trabajo práctico se desarrolla un File Transfer, usando como protocolo de transporte a UDP, pero con un protocolo intermediario en la capa de aplicación para asegurarnos de realizar una transferencia confiable. Se desarrolla una aplicación de tipo cliente-servidor, con las funcionalidades upload y download de archivos.

2. Hipótesis y suposiciones realizadas

- Los paquetes no llegan corrompidos (UDP cuenta con servicio de validación).
- Tener 10 timeouts significa una desconexión por lo que en caso de suceder esto se cortara la conexión automáticamente
- Se hace un manejo medido de envío de información lo que significa que no importa el tamaño del archivo la información nunca se enviara en bloques de mas de 5 Kbytes
- En caso de fallar la conexión al momento del handshake el usuario continuara intentando establecer una conexión exitosa

3. Implementación

Para el desarrollo del presente trabajo se implementó tanto un protocolo puramente de la capa de aplicación, como un protocolo que funcionara sobre UDP para poder convertirlo en un protocolo que provea *Reliable Data Transfer*. Nos referiremos a los dos protocolos como **Protocolo de Transferencia de Datos** y **Protocolo RDT** respectivamente.

Los archivos `start-server.py`, `upload.py` y `download.py` hacen uso de los objetos que implementan el protocolo de transferencia de datos para mandar el archivo dependiendo de los parámetros ingresados, y estos a su vez hacen uso del protocolo RDT abstrayéndose de cómo funciona por debajo.

3.1. Protocolo RDT

El protocolo RDT implementado presenta varias partes, pero primero hablaremos acerca de cómo lo utiliza el usuario.

Tenemos una clase llamada `RDTP`, la cual implementa dos métodos:

- **accept**: Se utiliza para escuchar en un puerto por conexiones entrantes que quieran hacer el **handshake**. Lo utiliza el servidor.
- **connect**: Se utiliza para intentar establecer una conexión con algún socket escuchando en una dirección especificada. Es utilizado por el cliente.

Cualquiera de estos dos métodos, internamente, va a abrir otro thread en el que vivirá un objeto de la clase `RDTPStream` y nos va a devolver un objeto del tipo `RDTPStreamProxy`.

El `RDTPStream` es el que se va a encargar de ahora en más de hacer los `sendto` y `recvfrom` y de garantizar las propiedades de una transferencia de datos confiable. Por su parte, el `RDTPStreamProxy` es el objeto a través del cual el cliente se va a comunicar con el stream. El proxy y el stream se comunican a través de `pipes`, para que cada vez que el usuario haga un `send` de alguna cantidad de bytes, estos sean enviados al stream para que se encargue de transmitirlos a través de un pipe. Cuando la aplicación hace un `recv` sucede algo similar, el stream manda la información que le llegó por otro pipe. De forma similar, el proxy le puede indicar al stream cuando ya no va a mandar más mensajes.

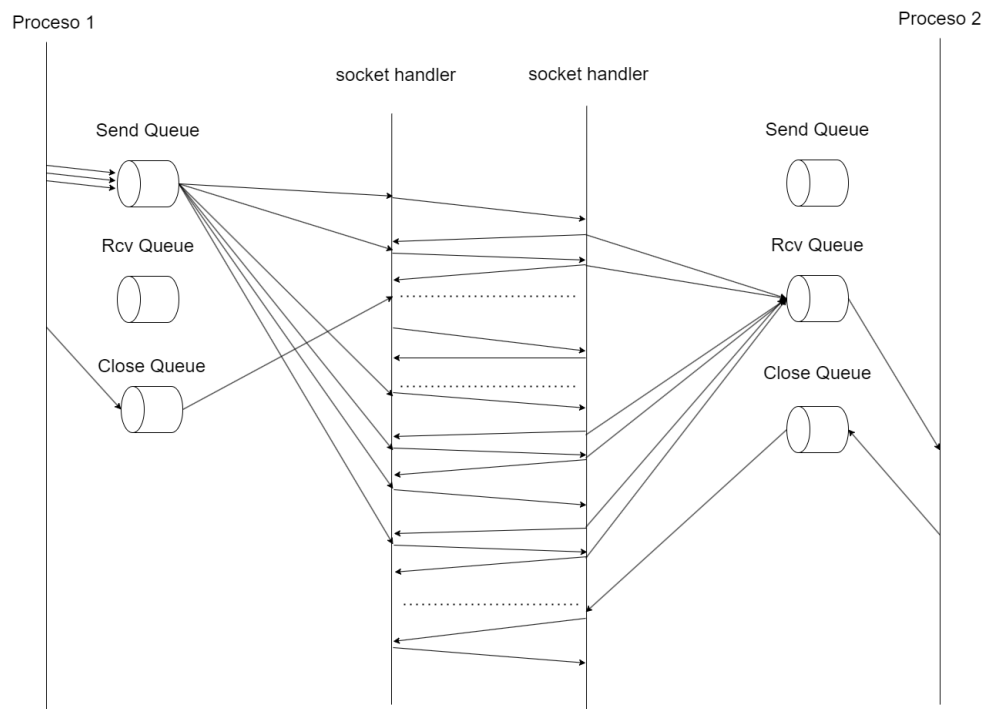


Figura 1: RDTP Stream y RDTP Stream Proxy

3.1.1. Segmentos

Para transmitir los mensajes a través de UDP y agregarles las características de un protocolo con transferencia confiable de datos, vamos a dividirlo en **Segmentos**, los cuales se componen de dos partes:

■ Header

- Tiene una longitud fija de 15 bytes.
- Cuenta con los siguientes campos:
 - **Source port**: puerto del emisor del mensaje (2 bytes).
 - **Destination port**: puerto destino del mensaje (2 bytes).
 - **Sequence number**: entero que representa el número del primer byte del mensaje que se está enviando (4 bytes).
 - **ACK number**: entero que indica el número del siguiente byte que estamos esperando (4 bytes).
 - **Data length**: tamaño en bytes del payload del mensaje (2 bytes).
 - **Flags**: Flags del mensaje (1 byte)

■ Payload

- Secuencia de bytes que representan una porción de los datos que el cliente está mandando.

Las diferentes combinaciones de flags nos van a indicar cómo tratar a los mensajes. Estás están representadas con un byte, pero al solo tener 4, solo utilizamos 4 bits. Estas son las siguientes:

■ PING:

- Indica que el presente es un mensaje de ping.
 - El mensaje no tiene payload
 - Cuando se recibe un mensaje con esta flag, se aumentará el ack number únicamente en 1.
- **SYN:**
 - Está presente en el primer mensaje que se manda al iniciar la conexión.
 - También está presente, en conjunción con la flag ACK, para reconocer un mensaje SYN previamente recibido.
 - El mensaje no tiene payload
 - **ACK:**
 - Indica que se está reconociendo la recepción de un mensaje
 - Nuestro protocolo no soporta piggyback, con lo cual si está flag está presente, el mensaje no tiene payload y se usa exclusivamente para confirmar la recepción de un mensaje, o transmitir que nos llegó un mensaje que no esperábamos.
 - **FIN:**
 - Indica la intención de una parte de la conexión de finalizarla.
 - Acompañada de la flag ACK, se usa para reconocer un mensaje de fin previamente enviado.

3.1.2. Establecimiento de la conexión (Handshake)

Previo al envío de datos, se realiza un handshake para que tanto cliente como servidor se preparen para el intercambio de información.

Es también en este momento que le damos al cliente un nuevo puerto por el cual se comunicará con el servidor de ahora en adelante. Esto lo hacemos debido a que el puerto conocido del servidor es el mismo que utilizarán todos los clientes para mandar mensajes de syn, debido a que UDP no diferencia por el puerto y dirección y nuestro servidor debe ser capaz de procesar solicitudes de múltiples clientes en paralelo.

El handshake consta de 3 etapas:

- Solicitud de conexión del cliente al servidor (mensaje SYN): el cliente envía al servidor un mensaje sin contenido en el payload, y con el flag SYN prendido. Esto indica que el cliente quiere establecer una conexión con el servidor.
- Aceptación de conexión del servidor al cliente (mensaje SYN-ACK): el servidor responde al mensaje SYN con otro mensaje sin contenido en el payload, y con los flags SYN y ACK encendidos. Esto quiere decir que recibió correctamente el mensaje SYN enviado por el cliente, y lo está reconociendo, por lo tanto está aceptando la conexión. Es en esta etapa que el servidor abre un nuevo socket y le envía al cliente en el campo source port el puerto nuevo por donde se deberá comunicar el cliente a partir de ese momento.
- Confirmación de la conexión (mensaje ACK-ACK): como último paso, el cliente le responde al servidor (en el nuevo puerto) con un último mensaje sin payload, con el flag ACK encendido. Dando a entender que recibió la confirmación de parte del servidor, por lo que ya pueden comenzar a enviarse información.

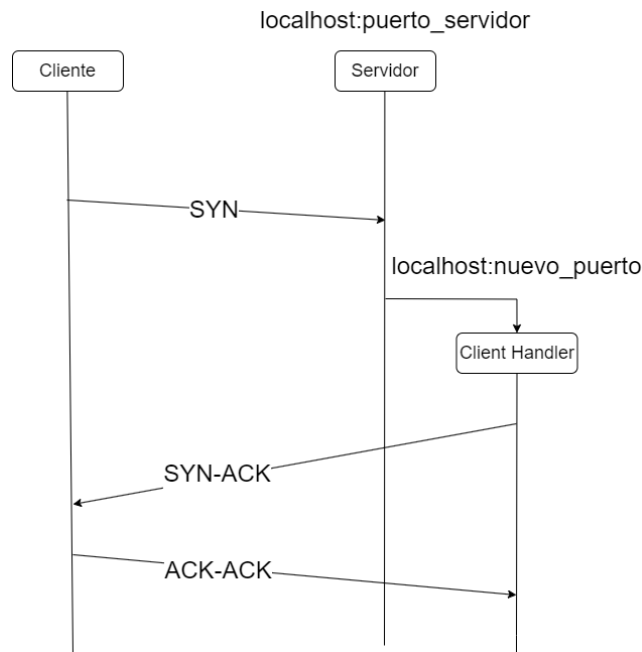


Figura 2: Proceso de handshake

El protocolo implementado no va a intentar mandar múltiples mensajes de syn ni de syn-ack, debido a que durante el handshake no sabemos si tenemos una conexión con el otro extremo, lo que llevaría a que esperar por ACKs, en especial el ACK-ACK, se vuelva algo que nunca termina, pues nunca sabrás a ciencia cierta si el mensaje que mandaste al final fue recibido, a menos que te quedes a esperar, y el otro lado haga lo mismo, sin salir de esta etapa.

Debido a esto, decidimos que se va a intentar hacer el handshake, y si ocurre algún timeout se va a considerar como un fracaso y volver a intentar desde el principio. Tanto el extremo que se conecta como el extremo al que se intentan conectar tienen un comportamiento que les permite saber si efectivamente tienen una conexión o el otro extremo en realidad consideró el handshake como fallido.

Después de establecido el handshake, cada extremo de la conexión va a tener un **RDTP Stream Proxy**, a través del cual se van a estar comunicando con un **RDTP Stream** que se encargará de mandar y recibir segmentos.

3.1.3. Pérdida de paquetes

El protocolo tiene en consideración la pérdida de paquetes a partir de haber establecido la conexión. Se estableció un único mecanismo para resolver la pérdida de paquetes, este fue establecer un timer por cada paquete enviado, y en el caso de que el timer supere un determinado tiempo que se llamo **TIMEOUT** se reenvía el paquete y se reinicia el timer.

Cabe destacar que el chequeo de que paquete reenviar por este timeout es el paquete más viejo que se envió pero que no se recibió la confirmación.

En el caso de haber ocurrido 10 timeouts, se interpretó como que el otro actor de la conexión se ha desconectado, y se cierra la conexión.

Una última consideración, en el caso de que se pierda la conexión, es decir, uno de los dos actores en la conexión por algún motivo se desconecte sin haber comunicado el fin de la conexión, se estableció un mecanismo para poder confirmar si el otro actor sigue, o no, conectado. Lo que se estableció son mensajes de pings, fuerza al actor a mandar un mensaje y por el mecanismo mencionado anteriormente, si el otro actor no responde a este ping, se interpreta que se ha desconectado.

Además del timeout para reenviar paquetes, si nos llegan paquetes con un número de secuencia que no esperábamos, reenviamos el último ACK message de lo que sí esperábamos.

Si recibimos un ACK de un mensaje del que ya habíamos recibido su ack, vamos a reenviar el último mensaje que mandamos y del que seguíamos esperando su ACK.

El comportamiento de los extremos con respecto a si guardar o no mensajes que les llegan en desorden dependerá del modo en el que se inicie al protocolo.

3.1.4. Stop and Wait

Nuestro protocolo puede actuar de manera stop and wait siguiendo el siguiente algoritmo:

- Se manda un único mensaje con un número de secuencia Q
- Se empieza un temporizador con un tiempo T
- Si nos llega un mensaje de ACK con un número ACK de Q entonces se integra a buffer de datos recibidos y se manda el siguiente mensaje.
- Si nos llega un mensaje de ACK con un número de ACK J o si se acaba el tiempo en el temporizador, se reenvía el mensaje con el número de secuencia Q.

3.1.5. Selective Repeat

Adicionalmente, nuestro protocolo tiene la funcionalidad de actuar de manera selective repeat. En este modo, tanto el que envía mensajes va a tener una **ventana** en la que guarde los mensajes que mandó para controlarlos y a medida que le llegan ACKs que pueden ser acumulativos, va mandando los siguientes segmentos para siempre tener la ventana llena.

Por otra parte, el que recibe mensajes tiene un buffer en el que guarda los mensajes que le llegan. A medida que va consiguiendo los que verdaderamente estaba esperando, se va a fijar en los mensajes del buffer para ver si tiene ahí el siguiente que espera, e integrarlo si es apropiado.

Curiosamente, al implementar el programa nos dimos cuenta que era equivalente la implementación de stop and wait, con selective repeat cuando se establece un tamaño de ventana de 1.

3.2. Protocolo de Transferencia de Datos

Una vez establecida la conexión con éxito, se hace el intercambio de datos. En primer lugar, el cliente envía un mensaje al servidor con la siguiente información en el payload:

- Stream: RDTPStreamProxy, como se recibe la información y el cliente se comunica y para obtenerla
- Action: la acción a realizar con el servidor, puede ser UPLOAD (0) de un archivo o DOWNLOAD (1) de un archivo (1 byte).
- File name: el nombre del archivo a descargar/cargar.
- File path: el path del archivo a descargar/cargar.
- Logger: Para notificar en cada momento el estado de la carga/descarga de datos

Luego de recibir el mensaje, el servidor ya sabe cómo actuar.

En caso de ser una descarga (el archivo se envía del servidor al cliente), el servidor enviará otro mensaje que contiene el tamaño del archivo a descargar. Caso contrario, será el cliente quien envíe el mensaje con el tamaño del archivo a cargar.

Algunas consideraciones a tener en cuenta:

Para el UPLOAD:

- El cliente sera el que envíe la información en este caso por lo que el servidor la estará recibiendo
- El cliente calcula el tamaño del archivo previamente a ser enviado ya que se tiene preestablecido una cantidad de datos a enviar. Si el archivo es muy grande, el cliente enviara los datos particionados, caso contrario lo hara completamente.
- Internamente el tamaño del archivo se ira reduciendo con los envios por lo que una vez que se llega a 0 significara que el archivo fue enviado en su totalidad

Para el DOWNLOAD:

- El servidor sera el que hace el envio de datos en este caso por lo que el cliente sera el que reciba la información
- En un primer lugar con la información del archivo que se recibe: dirección y nombre se intenta hacer el directorio donde se guardara el archivo descargado
- En ese mismo directorio ya creado, se hace la descarga al archivo
- Se hace de una forma casi identica al upload ya que con el tamaño del archivo que se recibe se hara la descarga por partes dependiendo del tamaño que tenga el archivo en al momento de la descarga

4. Pruebas

Vamos a analizar el rendimiento que tiene las aplicaciones, y por ende el protocolo implementado, sin y con packet loss.

Archivo	Stop and Wait		Selective Repeat	
	Upload [s]	Download [s]	Upload [s]	Download [s]
100 KB	0.53	0.49	0.58	0.50
1 MB	4.44	4.12	4.44	4.99
5 MB	22.33	20.89	26.40	31.13

Tabla 1: Valores obtenidos sin packet loss

Se puede notar que mientras más grande sea el tamaño del archivo, más lento es el Selective Repeat, lo que podemos suponer que la complejidad que presenta este se va haciendo más presente en el resultado.

Archivo	Stop and Wait		Selective Repeat	
	Upload [s]	Download [s]	Upload [s]	Download [s]
100 KB	0.52	1.57	0.69	0.73
1 MB	42.3	59.5	8.08	8.10
5 MB	247	243	42.3	45.1

Tabla 2: Valores obtenidos con 10 % packet loss

Se puede notar, ahora que se tiene packet loss, y también desorden de los paquetes, que la complejidad y optimizaciones del método de Selective Repeat son aprovechados, reduciendo aproximadamente 5 veces el tiempo de ejecución.

5. Preguntas a responder

Describe la arquitectura Cliente-Servidor.

En la arquitectura cliente-servidor existen dos componentes: un programa cliente y uno servidor. El cliente se ejecuta en un sistema terminal y es quien solicita y utiliza un servicio de un servidor, que se ejecuta en otro sistema terminal.

El servidor está siempre activo, esperando por solicitudes de distintos clientes. Los clientes realizan solicitudes al servidor cuando lo necesiten. El programa cliente y el servidor interactúan enviándose mensajes entre sí, habitualmente a través de Internet.

Dado que un programa cliente normalmente se ejecuta en una computadora y el programa servidor en otra, las aplicaciones cliente-servidor son aplicaciones distribuidas.

¿Cuál es la función de un protocolo de capa de aplicación?

La capa de aplicación es aquella que permite la comunicación entre los procesos de las aplicaciones a través de la red. Un protocolo de la capa de aplicación define cómo los procesos de una aplicación, que se ejecutan en distintos sistemas terminales, se envían los mensajes entre sí. En particular, un protocolo de la capa de aplicación define:

- los tipos de mensajes intercambiados: por ejemplo, mensajes de solicitud y mensajes de respuesta.
- la sintaxis de los diversos tipos de mensajes, es decir, los campos de los que consta el mensaje y cómo se delimitan esos campos.
- la semántica de los campos, es decir, el significado de la información contenida en los campos.
- las reglas para determinar cuándo y cómo un proceso envía mensajes y responde a los mismos.

La capa de transporte del stack TCP/IP ofrece dos protocolos: TCP y UDP. ¿Qué servicios proveen dichos protocolos? ¿Cuáles son sus características? ¿Cuándo es apropiado utilizar cada uno?

UDP ofrece casi lo mínimo que un protocolo de transporte debe hacer:

- Servicio de multiplexación y demultiplexación, para permitir la comunicación proceso a proceso.
- Control de integridad de los mensajes, usando el campo checksum.

En cambio, TCP ofrece varios servicios más, además de los ofrecidos por UDP:

- RDT (Reliable Data Transfer), esto asegura que los mensajes le llegan al host receptor en orden, sin corrupción en la información, sin errores ni duplicados.
- Control de flujo: TCP ajusta la velocidad a la que manda los mensajes para no sobrecargar el buffer receptor, evitando así la pérdida de paquetes.
- Control de congestión: ajusta la velocidad de transmisión de datos en función de las condiciones de la red, para no sobrecargarla si ya está exigida. Esto evita pérdida de paquetes y también es un mecanismo "solidario" (sirve a todos los que utilizan la red, porque evita problemas de rendimiento).

Es apropiado utilizar TCP cuando se necesita que los paquetes lleguen a su destino de forma completa y sin errores. En cuanto a UDP, es apropiado utilizarlo cuando se necesita que la información llegue de forma rápida y no sea muy significativo que exista pérdida de información.

6. Dificultades encontradas

La dificultad principal que se encontró en el transcurso de este trabajo fue bugs en el código, que gracias al uso de herramientas como wireshark pudimos detectar pero no necesariamente resolver inmediatamente.

Desde errores simples como al usar mininet, ver que no se podía establecer la conexión correctamente, y ver por wireshark que la información es correcta pero las direcciones usadas era lo que se estaba ingresando mal.

Hasta errores de tener un byte desplazado, que poder ver ese detalle ayudó corregir el error pero no ayudó a entender cual fue el error.

7. Conclusión

En conclusión podemos afirmar que este trabajo ayudo en gran medida a la impresión de la comunicación de los procesos a través de la red utilizando el modelo de servicio que la capa de transporte provee a la capa de aplicación.

Luego de varias pruebas y cambios en la implementación pudimos lograr una transferencia de datos con un protocolo propiamente creado siendo este de transferencia confiable, por eso el nombre RDTP (Reliable Data Transfer Protocol) que efectivamente pueda ser comunicada mediante el modelo de arquitectura cliente-servidor. Se desarrollo, con éxito, el protocolo RDTP sobre UDP: Stop-and-Wait y Selective Repeat cada uno con sus dificultades y diferencias detalladas a lo largo del informe.